

Milvus Vector Search Demo

Created by Mateusz Klocek

Introduction

This document will outline how I have used our chosen vector database library, [Milvus](#), to perform a vector search demo.

The implementation of Milvus vector search represents a significant leap forward in our ability to process and analyse complex datasets at scale, enabling faster, more accurate search capabilities that can fundamentally transform our approach to data retrieval and insights generation.

This document focuses on a simple yet powerful demonstration in 2 dimensions, setting the groundwork for more advanced applications and research.

Dataset

To perform a demo search, I created an example dataset to be able to perform the vector search on.

CSV exports of the datasets are available in our repository under the milvus branch inside the ai folder. *(These are only sample datasets, I have not made them with the intention of keeping them)*

Users Table Format:

user_id	email	password_hash	name	contact_number
9	simpsonmorgan@example.com	2834531c40cd18907597020f4c...	Kayla White	07923165210

Journey Table Format:

journey_id	address	user_id	start_latitude	start_longitude	departure_time
1	70A Ormiston Grove, London W12...	9 →	51.5088	-0.236265	07:00:00

For this demo, I used a mix of manually picking addresses and utilising Python to generate random data, and finally used MySQL to populate the database.

Demo Walkthrough

Code is available on Gitlab - Currently under development in the milvus branch inside the ai folder.

1. We begin by connecting to the Milvus Client on port 19530
2. Then we create a collection named 'journey_vectors' which we specify as having 2 dimensions.
3. We then fetch the journey data from the database, including the journey id, start latitude and start longitude.
4. Next, we format the data in such a way that Milvus will accept it.
5. We go ahead and insert the data into the collection.
6. Next, we perform the vector search.

To perform the search, we have been using the cosine similarity metric to measure similarities among the vectors:

$$\cos\theta = \frac{\sum_0^{n-1} (a_i \cdot b_i)}{\sqrt{\sum_0^{n-1} a_i^2} \cdot \sqrt{\sum_0^{n-1} b_i^2}}$$

7. We take the ID's from the search, and store them under *similar_journey_ids*.
8. Finally, we output the target users details, along with the search results.

Relevant Milvus Docs

For clear explanations of functions in the code please see the documentation:

Managing collections: <https://milvus.io/docs/manage-collections.md>

Inserting into collections: <https://milvus.io/docs/insert-update-delete.md>

Indexing into collections: <https://milvus.io/docs/index-vector-fields.md>

Single vector search: <https://milvus.io/docs/single-vector-search.md>

Steps Forward

Increase complexity of vector search

- Increase number of vectors to be able to include time of departure.
- Include filtering for search.

Research the different available similarity metrics:

- <https://milvus.io/docs/metric.md>

Research hybrid search:

- https://milvus.io/api-reference/pymilvus/v2.4.x/ORM/Collection/hybrid_search.md

Collaborate with the Backend Team

- Develop a comprehensive, well thought-out database schema tailored for storing our dataset effectively.